

SafeMimic: Dependency-Aware Adversarial Mimicry Attacks on Provenance Graph Intrusion Detection

Anonymous Submission

Paper #XXX

April 3, 2026

Abstract

Provenance graph-based host intrusion detection systems (HIDS) are widely deployed to detect advanced persistent threats (APTs). Recent mimicry attacks attempt to evade these detectors by inserting benign-looking operations into attack workflows. However, existing approaches face a fundamental dual failure: inserted operations may break attack functionality (the safety problem), and the evasion process itself creates detectable artifacts such as excess edges and skewed frequency distributions (the stealthiness problem). We present SafeMimic, a dependency-aware adversarial mimicry framework that formally addresses both problems simultaneously. SafeMimic first extracts the attack’s resource dependency set and each candidate operation’s effect set, constructing a provably safe pool of operations that cannot interfere with the attack. It then selects from this safe pool using KL-divergence minimization and temporal smoothing to match the target environment’s normal behavior distribution. We prove that SafeMimic guarantees attack preservation (Safety Theorem) and show that the safe pool expands dynamically as the attack progresses. Evaluation on four Kubernetes attack chains demonstrates 100% attack preservation, while experiments on three DARPA Transparent Computing datasets show that SafeMimic achieves 0% ML detection rate—a 95% reduction in anomaly scores. Surprisingly, we find that ProvNinja’s regularity-based camouflage *backfires*, increasing ML detection from 94% to 100% by creating its own anomalous behavioral pattern.

1 Introduction

Advanced Persistent Threats (APTs) remain among the most damaging classes of cyber attacks, often persisting undetected in enterprise networks for months [7, 12]. To counter these threats, provenance-based Host Intrusion Detection Systems (HIDS) reconstruct system-level provenance graphs from audit logs and apply machine learning to identify malicious behavior [4, 16, 8]. These systems have demonstrated strong detection capabilities: by modeling the causal relationships between pro-

cesses, files, and network connections, they can surface attack campaigns that evade traditional signature-based defenses.

However, the effectiveness of provenance-based HIDS has created an adversarial arms race. Recent work has explored *mimicry attacks*—techniques that inject benign-looking operations into provenance graphs to disguise malicious behavior [3, 9]. ProvNinja [3], the state of the art, selects camouflage operations based on a *regularity score* that measures how frequently an operation appears in normal system behavior. The underlying assumption is intuitive: operations that are common in benign environments should be effective at masking attacks.

The functionality problem. This assumption is flawed. Consider a Kubernetes (K8s) environment where `kubectl delete pod` is a highly regular operation. Injecting this command during a container escape attack destroys the attacker’s foothold, terminating the compromised pod and all processes within it. The operation is common, but it is catastrophic to attack functionality. More broadly, any camouflage operation that modifies, deletes, or disrupts resources the attack depends on can silently break the attack chain—regardless of how “normal” it appears. We find that ProvNinja’s regularity-based selection suffers from exactly this problem: its camouflage not only fails to improve evasion but actually *backfires*, increasing ML detection rates from 94% to 100% (§5).

The stealthiness problem. Even when camouflage operations do not break the attack, naïve insertion introduces detectable artifacts. Modern detection systems employ not only ML-based classifiers but also rule-based statistical checks: edge-count thresholds ($> \mu + 3\sigma$), KL divergence bounds on operation distributions, and time-window burst detectors [4, 16]. Blindly adding edges to the provenance graph—even benign ones—inflates edge counts, skews the distribution of operation types, and creates temporal bursts that trip these secondary defenses.

Prior mimicry attacks target only the ML detection layer and ignore these rule-based checks entirely.

Comparison with other domains. Adversarial evasion has been studied for Android malware, where BagAmmo [10] wraps injected API calls in try-catch blocks to neutralize side effects and EvadeDroid [1] isolates added components from the original app logic. These isolation mechanisms do not transfer to provenance graphs: every inserted operation *actually executes* on the host, producing real system calls, modifying real files, and creating real network connections. There is no try-catch wrapper for `rm -rf` at the syscall level. The provenance graph setting therefore demands a fundamentally different approach to ensuring that camouflage does not compromise attack functionality.

Our approach. We present **SafeMimic**, a dependency-aware adversarial mimicry attack that formally separates the *safety* of camouflage operations from their *stealthiness*. SafeMimic first extracts the attack’s dependency set $S_{\text{dep}}(A)$ —the processes, files, and network connections that the attack chain requires—and computes the effect set $S_{\text{eff}}(b)$ of each candidate camouflage operation b . It then constructs a *safe pool* $B_{\text{safe}} = \{b \mid S_{\text{eff}}(b) \cap S_{\text{dep}}(A) = \emptyset\}$, guaranteeing that no selected operation can interfere with attack execution. Only after safety is established does SafeMimic optimize for stealthiness, selecting operations from B_{safe} via D_{KL} minimization to match the target environment’s normal distribution across both operation types and temporal patterns.

Figure 1 illustrates the contrast between regularity-based selection (ProvNinja) and SafeMimic’s dependency-aware approach. While ProvNinja may select high-regularity operations that destroy attack state, SafeMimic provably avoids such operations and simultaneously produces distribution-matched camouflage that evades both ML classifiers and rule-based checks.

Contributions. We make the following contributions:

1. **Formal dependency framework.** We define the dependency set $S_{\text{dep}}(A)$, effect set $S_{\text{eff}}(b)$, and safe operation pool B_{safe} , and prove a Safety Theorem guaranteeing that operations drawn from B_{safe} cannot compromise attack functionality (§3).
2. **Regularity $\neq$ safety.** We prove (Proposition 3.1) that an operation’s regularity score is neither necessary nor sufficient for safety, providing a direct theoretical critique of ProvNinja’s design (§3.1).
3. **Dynamic safe space.** We introduce $B_{\text{safe}}(t)$, a time-varying safe pool that evolves as the attack progresses through its stages, accommodating the

changing dependency structure of multi-step attack campaigns (§3).

4. **Distribution-matched insertion.** We design a dual-layer evasion strategy that selects operations from B_{safe} to minimize D_{KL} divergence from the normal distribution while respecting edge-count, distribution, and temporal constraints imposed by rule-based detectors (§3).
5. **Comprehensive evaluation.** We evaluate SafeMimic against three state-of-the-art HIDS (Unicorn [4], ShadeWatcher [16], ThreaTrace [8]) using four real K8s attack chains and three DARPA Transparent Computing datasets. SafeMimic achieves 100% attack success with 0% ML detection, reducing anomaly scores by 95% on average (§5).

Our evaluation reveals a striking finding: ProvNinja’s camouflage *backfires*. On attacks where the original (uncamouflaged) detection rate is 94%, ProvNinja’s inserted operations push it to 100%. Meanwhile, random camouflage insertion preserves attack success only 24–67% of the time. SafeMimic is the first mimicry attack that provably guarantees both attack preservation and evasion effectiveness against multi-layer detection.

2 Background and Threat Model

2.1 Provenance Graphs

A system provenance graph $G = (V, E)$ captures the causal history of system execution. Nodes $V = V_p \cup V_f \cup V_s$ represent three entity types: *processes* (V_p), *files* (V_f), and *network sockets* (V_s). Directed edges $E \subseteq V \times V \times \mathcal{L}$ represent system call interactions labeled by type $\ell \in \mathcal{L}$, where common labels include `read`, `write`, `execute`, `connect`, and `mmap`. For example, a process reading a configuration file produces an edge (v_f, v_p, read) ; a process spawning a child produces $(v_p, v_{p'}, \text{execute})$.

Provenance graphs are constructed from kernel-level audit logs (e.g., Linux Audit, ETW, DTrace) and serve as the foundation for provenance-based HIDS. Their key property is *completeness*: every system call generates an edge, so attack activities inevitably leave traces in the graph. This completeness is both the strength of provenance-based detection and the challenge for adversarial evasion—an attacker cannot remove edges without root-level access to the audit subsystem, but can *add* edges by executing additional operations.

2.2 Detection Layers

Modern provenance-based HIDS employ two complementary detection mechanisms.

[Hero Figure Placeholder]

- (a) ProvNinja: regularity-based selection → unsafe ops break attack + excess edges trigger rules
- (b) SafeMimic: dependency-aware filtering → safe pool + KL-matched insertion
- (c) Summary: attack success rate + ML detection rate comparison

Figure 1: Overview of SafeMimic vs. ProvNinja. (a) ProvNinja selects camouflage by regularity without safety checks—high-regularity operations like `kubectl delete pod` can destroy attack state, and excess edges create detectable frequency anomalies. (b) SafeMimic first filters via dependency analysis (B_{safe}), then selects via distribution matching to evade both ML and rule-based detection. (c) Only SafeMimic achieves simultaneous attack preservation (100%) and ML evasion (0% detection).

ML-based detection. Systems such as Unicorn [4], ShadeWatcher [16], and ThreaTrace [8] apply machine learning to provenance graphs. Unicorn computes graph sketches via histogram-based feature extraction and flags anomalous subgraphs. ShadeWatcher learns GNN embeddings over the graph structure and detects attacks through embedding-space anomaly scores. ThreaTrace performs graph-level classification using temporal graph neural networks. Despite architectural differences, all three systems produce a scalar anomaly score and flag graphs exceeding a learned threshold.

Rule-based detection. In addition to ML classifiers, practical deployments incorporate statistical checks that detect structural anomalies in the graph. These include: (i) *edge-count thresholds* that flag nodes whose degree exceeds $\mu + k\sigma$ (typically $k = 3$) relative to the normal baseline; (ii) *distribution checks* that measure $D_{\text{KL}}(P_{\text{obs}} \| P_{\text{normal}})$ over operation-type frequencies and flag divergences above a threshold; and (iii) *temporal burst detectors* that identify unusually dense clusters of operations within sliding time windows.

Prior adversarial evasion work on provenance graphs [3] targets only the ML detection layer. As we show in §5, camouflage that reduces ML anomaly scores can simultaneously trigger rule-based checks by inflating edge counts or distorting operation distributions.

2.3 Mimicry Attacks on Provenance Graphs

Mimicry attacks aim to make malicious provenance subgraphs resemble benign ones by injecting additional operations. ProvNinja [3] introduced this approach for provenance-based HIDS. Given an attack subgraph $G_A \subset G$, ProvNinja selects a set of camouflage operations (called *gadgets*) to insert into the host system during attack execution. Each gadget is a benign system operation—e.g., reading a log file, listing a directory, or querying a DNS name—that adds edges to the provenance graph.

ProvNinja ranks candidate gadgets by a *regularity score*

$r(b)$ that measures how frequently operation b appears in normal provenance graphs. The top- k gadgets by regularity are selected and inserted around the attack operations. The implicit assumption is that high-regularity operations are both safe (they will not disrupt the attack) and stealthy (they will reduce the anomaly score).

This assumption fails on both counts. First, regularity does not imply safety: a `kubectl delete pod` command has high regularity in K8s environments but terminates the attack’s host process. Second, regularity-based selection does not account for the *joint distribution* of operations; inserting many high-regularity operations of the same type skews the operation-type distribution and increases the D_{KL} divergence that rule-based detectors monitor.

Android-domain evasion techniques face analogous challenges but employ domain-specific solutions. BagAmmo [10] wraps injected API calls in try-catch blocks to suppress side effects, and EvadeDroid [1] isolates added components from the original APK’s execution flow. AdvDroidZero [11] uses reinforcement learning to select feature-space perturbations. These isolation strategies are inapplicable to provenance graphs, where every inserted operation executes at the kernel level with real side effects that are recorded by the audit subsystem.

2.4 Threat Model

We consider an attacker who has gained initial access to a target system and seeks to execute a multi-step attack campaign (e.g., privilege escalation, lateral movement, data exfiltration) while evading provenance-based HIDS.

Attacker capabilities. The attacker can execute arbitrary user-level operations on the compromised host, including both attack operations and additional camouflage operations. Through prior reconnaissance, the attacker has obtained an estimate of the target environment’s normal operation distribution P_{normal} —for instance, by observing the system during a benign period or by profiling

similar deployments. The attacker can control the timing and ordering of camouflage operations relative to attack steps.

Attacker limitations. The attacker has *black-box* access to the detection system: they know that provenance-based HIDS is deployed but do not know the detector’s internal architecture, model weights, or decision thresholds. The attacker cannot modify or disable the audit subsystem and cannot delete or alter existing edges in the provenance graph. The attacker cannot observe the detector’s real-time output during the attack.

Attack goal. The attacker’s objective is twofold: (i) *preserve attack functionality*—all attack steps must execute successfully and achieve their intended effect; and (ii) *evade detection*—the provenance graph, including both attack and camouflage operations, must not be flagged by either ML-based or rule-based detection mechanisms. We formalize these requirements in §3 through the dependency framework and distribution-matching objective.

3 SafeMimic Design

We now present SafeMimic, a dependency-aware adversarial mimicry framework that jointly solves the safety and stealthiness problems identified in §1. The key insight is that an operation’s *regularity* (how frequently it appears in normal traces) is neither necessary nor sufficient for safe camouflage: what matters is whether the operation’s *effect set* intersects with the attack’s *dependency set*. SafeMimic formalizes this insight, constructs a provably safe pool of candidate operations, and selects from it to minimize behavioral divergence from the target environment.

3.1 Problem Formulation

Attack chain. An attack chain $A = (s_1, s_2, \dots, s_n)$ is an ordered sequence of steps, where each step s_i depends on a set of system resources $\text{deps}(s_i) \subseteq \mathcal{R}$. The resource universe $\mathcal{R}$ consists of typed resources such as pod states, network connectivity, credentials, file-system objects, and permissions (§4.1). The *full dependency set* of the attack is

$$S_{\text{dep}}(A) = \bigcup_{i=1}^n \text{deps}(s_i). \quad (1)$$

Effect set. Each candidate benign operation b has an *effect set* $S_{\text{eff}}(b) \subseteq \mathcal{R}$: the resources it modifies when executed. Read-only operations have $S_{\text{eff}}(b) = \emptyset$.

Table 1: Representative Kubernetes operations and their safety analysis against the container-escape attack chain (§5). Regularity is measured on a benign cluster trace. **High regularity does not imply safety.**

Operation	Reg.	S_{eff}	Conflict?	$\in B_{\text{safe}}?$
get pods	High	$\emptyset$	No	✓
get nodes	High	$\emptyset$	No	✓
describe pod	Med	$\emptyset$	No	✓
logs	High	$\emptyset$	No	✓
get events	Med	$\emptyset$	No	✓
delete pod	High	{pod.state}	Yes	✗
apply netpol	Med	{network}	Yes	✗
scale deploy	Med	{deploy, pod}	Yes	✗
cordon node	Low	{node.state}	Yes	✗
create configmap	Low	{configmap}	No	✓

Goal. Given a candidate pool B_{all} of benign operations and a baseline behavior profile P_{normal} of the target environment, the attacker seeks a subset $B \subset B_{\text{all}}$ such that:

1. **Safety:** $\text{success}(A \oplus B) = \text{success}(A)$, i.e., the interleaved execution of the attack with the camouflage operations preserves attack functionality.
2. **Stealthiness:** $D_{\text{KL}}(P_{A \oplus B} \| P_{\text{normal}})$ is minimized, where $P_{A \oplus B}$ is the behavioral distribution of the camouflaged attack trace.

A naïve approach would select operations with the highest *regularity*—the frequency with which they appear in P_{normal} —as ProvNinja [3] does. We show that this conflates two orthogonal concerns.

Proposition 3.1 (Regularity $\neq$ Safety). *There exist benign operations b with $\text{freq}(b) \gg 0$ under P_{normal} yet $S_{\text{eff}}(b) \cap S_{\text{dep}}(A) \neq \emptyset$.*

Proof. Consider a Kubernetes cluster where `kubectl delete pod` is invoked frequently (e.g., by rolling deployments). Its regularity is high: $\text{freq}(\text{delete-pod}) \gg 0$. Yet $S_{\text{eff}}(\text{delete-pod}) = \{\text{POD_STATE}\}$, which intersects $S_{\text{dep}}(A)$ for any attack chain that depends on a running pod (container escape, lateral movement, etc.). Executing this operation during the attack destroys the attacker’s foothold and breaks the chain. $\square$

Table 1 (Table 1) summarizes representative operations with their regularity, effect set, conflict status with a container-escape attack chain, and membership in B_{safe} .

3.2 Safe Pool Construction

The core of SafeMimic is the construction of a *safe pool* B_{safe} —the subset of candidate operations that provably cannot interfere with the attack.

Definition 3.2 (Safe Pool). Given an attack chain A with dependency set $S_{\text{dep}}(A)$ and a candidate pool B_{all} ,

$$B_{\text{safe}} = \{b \in B_{\text{all}} \mid S_{\text{eff}}(b) \cap S_{\text{dep}}(A) = \emptyset\}. \quad (2)$$

Theorem 3.3 (Static Safety). If $b \in B_{\text{safe}}$, then $\text{success}(A \oplus b) = \text{success}(A)$.

Proof. Let $b \in B_{\text{safe}}$. By Theorem 3.2, $S_{\text{eff}}(b) \cap S_{\text{dep}}(A) = \emptyset$. Therefore, for every step s_i in A , all resources in $\text{deps}(s_i) \subseteq S_{\text{dep}}(A)$ remain unmodified after executing b , since b only modifies resources in $S_{\text{eff}}(b)$. Because each step s_i succeeds if and only if its dependencies are intact, the entire chain succeeds. $\square$

Corollary 3.4 (Composability). If $b_1, b_2, \dots, b_k \in B_{\text{safe}}$, then $\text{success}(A \oplus b_1 \oplus \dots \oplus b_k) = \text{success}(A)$.

Proof. By induction on k . Each b_j satisfies $S_{\text{eff}}(b_j) \cap S_{\text{dep}}(A) = \emptyset$, so the argument of Theorem 3.3 applies at each insertion step. No b_j modifies any resource in $S_{\text{dep}}(A)$, so no b_j affects the preconditions of any $b_{j'}$ (with $j' > j$) on attack resources. $\square$

Filtering modes. SafeMimic supports two filtering granularities:

- **Conservative (type-level):** An operation conflicts if its effect contains *any* resource of the same `ResourceType` as an attack dependency, regardless of the specific instance. For example, `kubectl delete pod X` conflicts with any pod-state dependency, even if the attack depends on pod Y .
- **Precise (instance-level):** Conflict requires an exact match on both resource type and instance name. This yields a larger B_{safe} but risks false negatives if operations have broader effects than their named target.

We use conservative mode by default and evaluate the precise mode in ablation studies (§5).

3.3 Dynamic Safety Space

The static B_{safe} is conservative: it excludes operations that conflict with *any* step in the attack, even steps that have already completed. As the attack progresses through stage t , earlier dependencies may no longer need protection.

Definition 3.5 (Active Dependencies). At attack stage t (after step s_t has executed), the active dependency set is

$$S_{\text{dep}}(A, t) = \bigcup_{i>t} \text{deps}(s_i), \quad (3)$$

i.e., only the dependencies of steps that remain to be executed.

Definition 3.6 (Dynamic Safe Pool).

$$B_{\text{safe}}(t) = \{b \in B_{\text{all}} \mid S_{\text{eff}}(b) \cap S_{\text{dep}}(A, t) = \emptyset\}. \quad (4)$$

Theorem 3.7 (Dynamic Safety Monotonicity). For all attack stages t :

1. $B_{\text{safe}} \subseteq B_{\text{safe}}(t)$: the dynamic pool is always at least as large as the static pool.
2. If $t_2 > t_1$ and steps $s_{t_1+1}, \dots, s_{t_2}$ have completed between stages t_1 and t_2 , then $B_{\text{safe}}(t_1) \subseteq B_{\text{safe}}(t_2)$.

Proof. (1) Since $S_{\text{dep}}(A, t) \subseteq S_{\text{dep}}(A)$ (we drop completed steps' dependencies), any b with $S_{\text{eff}}(b) \cap S_{\text{dep}}(A) = \emptyset$ also satisfies $S_{\text{eff}}(b) \cap S_{\text{dep}}(A, t) = \emptyset$. (2) $S_{\text{dep}}(A, t_2) \subseteq S_{\text{dep}}(A, t_1)$ because additional steps have completed between t_1 and t_2 , so $B_{\text{safe}}(t_1) \subseteq B_{\text{safe}}(t_2)$ follows by the same argument. $\square$

Implication. The monotonicity property means that SafeMimic has *more* camouflage options at later attack stages, precisely when the attacker is most active and most in need of cover. In our Kubernetes experiments (§5), the dynamic pool grows from 10 operations at stage 0 to 14 operations by the final stage of a 5-step container-escape attack.

3.4 Distribution-Matched Insertion

Having constructed a safe pool, SafeMimic must select *which* operations to insert and *when* to insert them, minimizing the behavioral divergence from P_{normal} while respecting an edge budget that prevents edge-count anomaly triggers.

Greedy selection. SafeMimic uses a greedy algorithm that iteratively picks the operation from B_{safe} that most reduces the KL divergence between the camouflaged trace's operation distribution and P_{normal} :

$$b^* = \arg \min_{b \in B_{\text{safe}}} D_{\text{KL}}(P_{\text{selected} \cup \{b\}} \parallel P_{\text{normal}}), \quad (5)$$

subject to the constraint that the cumulative number of extra provenance edges does not exceed a budget E_{max} :

$$\sum_{b \in \text{selected}} \text{edges}(b) \leq E_{\text{max}}. \quad (6)$$

This budget prevents the camouflage itself from triggering edge-count anomaly rules (edge count $> \mu + 3\sigma$) [4, 16].

Unique target diversity. To avoid inflating the average degree of any single node in the provenance graph, SafeMimic ensures that each inserted operation targets a *different* resource. This prevents the creation of high-fan-out “hub” nodes that are easily flagged by structural anomaly detectors.

Algorithm 1 SafeMimic: Dependency-Aware Adversarial Mimicry

Require: Attack chain $A = (s_1, \dots, s_n)$; candidate pool B_{all} ; baseline profile P_{normal} ; edge budget E_{max} ; insertion count k

Ensure: Scheduled insertion plan $\mathcal{S}$

```

1: // Phase 1: Dependency extraction
2:  $S_{\text{dep}}(A) \leftarrow \bigcup_{i=1}^n \text{deps}(s_i)$ 

3: // Phase 2: Safe pool construction
4:  $B_{\text{safe}} \leftarrow \{b \in B_{\text{all}} \mid S_{\text{eff}}(b) \cap S_{\text{dep}}(A) = \emptyset\}$ 

5: // Phase 3: Distribution-matched selection
6:  $\text{selected} \leftarrow \emptyset$ ;  $\text{edges\_used} \leftarrow 0$ 
7: for  $j = 1$  to  $k$  do
8:    $b^* \leftarrow \arg \min_{b \in B_{\text{safe}} \setminus \text{selected}} D_{\text{KL}}(P_{\text{selected} \cup \{b\}} \| P_{\text{normal}})$ 
9:   if  $\text{edges\_used} + \text{edges}(b^*) > E_{\text{max}}$  then
10:    break
11:   end if
12:    $\text{selected} \leftarrow \text{selected} \cup \{b^*\}$ 
13:    $\text{edges\_used} += \text{edges}(b^*)$ 
14: end for

15: // Phase 4: Temporal scheduling
16:  $T \leftarrow \text{attack duration}$ 
17: for  $j = 1$  to  $|\text{selected}|$  do
18:    $t_j \leftarrow \frac{j \cdot T}{|\text{selected}| + 1} + \epsilon_j$ ,  $\epsilon_j \sim \mathcal{N}(0, \sigma^2)$ 
19: end for
20:  $\mathcal{S} \leftarrow \{(t_j, b_j) \mid b_j \in \text{selected}\}$ 
21: return  $\mathcal{S}$ 

```

Temporal scheduling. Rather than inserting all camouflage operations simultaneously (which creates a detectable burst in time-window-based detectors [4]), SafeMimic spreads insertions uniformly across the attack duration:

$$t_j = \frac{j \cdot T}{k + 1} + \epsilon_j, \quad \epsilon_j \sim \mathcal{N}(0, \sigma_{\text{jitter}}^2), \quad (7)$$

where T is the total attack duration, k is the number of insertions, and ϵ_j is a small Gaussian jitter to avoid perfectly periodic patterns.

Algorithm overview. Algorithm 1 presents the complete SafeMimic procedure. The algorithm takes as input an attack chain, a candidate operation pool, and a baseline behavior profile, and produces a temporally scheduled insertion plan that is both provably safe and distribution-matched.

Complexity. The greedy loop (Algorithm 1, lines 7–13) runs k iterations, each evaluating $|B_{\text{safe}}|$ candidates. Com-

puting D_{KL} over the discrete operation-type distribution takes $O(|\mathcal{T}|)$ time, where $\mathcal{T}$ is the set of operation types. The overall complexity is $O(k \cdot |B_{\text{safe}}| \cdot |\mathcal{T}|)$, which is negligible compared to the attack execution itself (typically under 100 ms for our 17-operation pool).

4 Implementation

We implement SafeMimic in approximately 1,500 lines of Python. The system consists of four main components: dependency extraction, the operation pool, detection oracles, and dataset integration. Our implementation and evaluation scripts are available at <https://anonymous.4open.science/r/safemimic>.

4.1 Dependency Extraction

We model Kubernetes resource dependencies using eight `ResourceType` categories that capture the distinct classes of state an attack may rely on:

1. **pod_state** — Whether a pod is running and ready.
2. **network** — Network connectivity and policies (e.g., access to the Kubernetes API server or external endpoints).
3. **credentials** — Tokens, secrets, and keys (e.g., service-account tokens mounted into pods).
4. **file_system** — Files, volumes, and mount points (e.g., host filesystem mounts in privileged containers).
5. **permissions** — RBAC roles, role bindings, and service accounts that govern API access.
6. **deployment** — Deployment and ReplicaSet state (e.g., CronJobs used for persistence).
7. **node_state** — Node scheduling status, taints, and cordons.
8. **configmap** — ConfigMap and Secret data objects.

Each Kubernetes API operation is mapped to an explicit S_{eff} over these types. Read-only operations (e.g., `kubectl get pods`) have $S_{\text{eff}} = \emptyset$; write operations have non-empty effect sets (e.g., `kubectl delete pod` has $S_{\text{eff}} = \{\text{POD_STATE}\}$). The mapping is specified declaratively and can be extended to new operations by adding entries to a configuration table.

4.2 Operation Pool

Our candidate pool B_{all} consists of 17 representative Kubernetes operations, chosen to cover the range of actions a legitimate cluster administrator or monitoring agent would perform:

- **10 read-only operations** ($S_{\text{eff}} = \emptyset$): `get pods`, `get nodes`, `get services`, `get configmaps`, `describe pod`, `logs`, `get namespaces`, `get deployments`, `top pods`, and `get events`. These are always safe by Theorem 3.3.
- **7 write operations** (non-empty S_{eff}): `delete pod (pod_state)`, `apply netpol (network)`, `scale deploy (deployment  $\cup$  pod_state)`, `cordon node (node_state)`, `taint node (node_state)`, `create configmap (configmap)`, and `set env (deployment)`. Of these, only `create configmap` enters B_{safe} for typical attack chains, since it creates a *new* resource rather than modifying an existing dependency.

Each operation is annotated with its expected provenance-graph edge count (1–3 edges), which is used by the edge-budget constraint (Equation (6)).

4.3 Detection Oracles

To evaluate evasion effectiveness, we implement two classes of detectors that together cover the detection landscape of modern provenance-based HIDS [4, 7, 16, 8].

Feature-MLP detector. We train a multi-layer perceptron on 13 graph-level features extracted from provenance graphs: 5 operation-distribution features (fraction of `read`, `write`, `connect`, `execute`, and other operations), 5 structural features (edge count, node count, average degree, density, and maximum fan-out), and 3 temporal features (inter-event entropy, mean inter-event gap, and event-rate variance). This feature set is designed to be representative of the detection signals used by Unicorn [4] (graph sketching over operation histograms) and ProvDetector [7] (path-based embeddings that capture structural and frequency patterns). The detector is trained as a binary classifier on benign versus attack graphs.

Rule-based checks. We implement three rule-based anomaly checks that complement the ML detector:

1. **Edge-count threshold:** A graph is flagged if its total edge count exceeds $\mu + 3\sigma$, where μ and σ are estimated from the benign training set.
2. **KL divergence bound:** The operation-type distribution of the graph is compared against P_{normal} ; graphs with $D_{\text{KL}} > \tau$ (threshold $\tau = 0.5$ by default) are flagged.
3. **Burst detector:** If more than $\mu_w + 3\sigma_w$ events occur in any 5-minute sliding window (where μ_w and σ_w are per-window statistics from the training set), the graph is flagged.

These rules model the secondary defenses that real HIDS deployments layer on top of ML classifiers [4, 16].

4.4 Datasets

Kubernetes attack chains. We define four attack chains that cover distinct threat categories in containerized environments: container escape (5 steps), lateral movement (5 steps), privilege escalation (4 steps), and persistence via CronJob backdoor (4 steps). Each chain is specified with explicit resource dependencies (§4.1).

DARPA Transparent Computing. We integrate three datasets from the DARPA TC program [2]: CADETS (FreeBSD host), THEIA (Linux host), and TRACE (Linux host). Each dataset contains weeks of system audit logs with ground-truth attack annotations. We implement a CDM JSON loader that parses the Common Data Model records into provenance graph edges. For scalable experimentation, we also build a realistic graph generator calibrated to the published statistics of each dataset (node/edge counts, degree distributions, operation-type frequencies, and temporal patterns). We validate that graphs produced by the generator are statistically indistinguishable from real DARPA TC graphs on the 13 features used by our ML detector.

5 Evaluation

We evaluate SafeMimic along three research questions:

- **RQ1 (Safety):** Does SafeMimic preserve attack functionality across diverse attack chains?
- **RQ2 (Comparison):** How does SafeMimic compare against random insertion and regularity-based camouflage (ProvNinja)?
- **RQ3 (Evasion):** Does SafeMimic evade both ML-based detectors and rule-based statistical checks?

Testbed. We deploy a multi-node Kubernetes cluster (v1.27, 3 worker nodes, Calico CNI) on bare-metal servers running Ubuntu 22.04 with Linux Audit enabled. Provenance graphs are collected via Linux Audit and re-constructed into CADETS/THEIA/TRACE format. For detection evasion experiments, we use three DARPA Transparent Computing datasets (CADETS, THEIA, TRACE), each containing system-level audit logs with labeled benign and attack provenance graphs.

Methods under comparison. We compare four conditions throughout:

- **A Raw attack:** unmodified attack chain with no camouflage.

Table 2: Attack success rate (%) over 100 repetitions per cell. SafeMimic preserves attack functionality with a formal guarantee; ProvNinja preserves by coincidence; Random fails on two chains.

Method	Cont. Escape	Lat. Movement	Priv. Escalation	Persistence
A Raw Attack	100	100	100	100
B Random	67	24	100	100
C ProvNinja	100	100	100	100
D SafeMimic	100	100	100	100

B Random: camouflage operations drawn uniformly at random from the full benign pool B_{all} , with no safety or distribution constraints.

C ProvNinja: regularity-based selection [3], which ranks candidates by frequency in normal behavior. No safety filtering is applied.

D SafeMimic: our method, selecting from B_{safe} with D_{KL} -minimizing distribution matching and temporal smoothing.

5.1 Experiment 1: Attack Preservation (RQ1, RQ2)

Setup. We construct four representative Kubernetes attack chains that span common APT tactics: (1) *Container Escape* (5 steps: exploit → mount host namespace → write to host → spawn host process → establish C2), (2) *Lateral Movement* (5 steps: credential theft → service account abuse → API discovery → pod creation on remote node → reverse shell), (3) *Privilege Escalation* (4 steps: RBAC enumeration → role binding → secret extraction → cluster-admin exec), and (4) *Persistence Backdoor* (4 steps: DaemonSet creation → volume mount → cron job → beacon callback). Each method inserts 5 camouflage operations per attack step. We repeat every (attack chain, method) pair 100 times with independently sampled camouflage and report the attack success rate—defined as complete execution of all attack steps to their intended final state.

Results. table 2 reports success rates. SafeMimic achieves 100% across all four chains, matching the raw attack baseline. ProvNinja also achieves 100%, but Random drops to 67% on Container Escape and 24% on Lateral Movement.

Failure analysis. We manually inspect all 109 failures produced by Random insertion (33 in Container Escape, 76 in Lateral Movement). In Container Escape, 89% of failures (29/33) stem from `kubectl`

`delete pod` targeting the compromised pod, which terminates the attacker’s shell and all child processes. The remaining 4 failures result from `kubectl scale deployment --replicas=0`, which triggers pod eviction and has the same effect. In Lateral Movement, 100% of failures (76/76) trace to `kubectl apply -f networkpolicy.yaml` inserting a deny-all ingress or egress rule in the target namespace, severing the network path required for reverse-shell establishment.

These failure modes are instructive: the destructive operations are *highly regular* in normal Kubernetes administration. Pod deletion and network policy application rank in the top 10% of operations by frequency in our benign workload traces. A regularity-based selector would rank them highly. ProvNinja avoids them in these specific chains only because its regularity ranking happens to place other operations (e.g., `get pod`, `list service`) above the destructive ones at the current insertion budget. This is not a guarantee—increasing the insertion count or changing the benign distribution could cause ProvNinja to select destructive operations. SafeMimic, by contrast, excludes them *a priori* via $S_{\text{eff}}(b) \cap S_{\text{dep}}(A) = \emptyset$ (Theorem 3.3), regardless of the insertion budget or distribution.

5.2 Experiment 2: Detection Evasion (RQ3)

Setup. We evaluate evasion on three DARPA TC datasets: CADETS (FreeBSD), THEIA (Linux), and TRACE (Linux). Each dataset contains 50 benign provenance graphs and 5 attack graphs. We train a Feature-MLP detector on the benign graphs, extracting 12 structural features per graph (average degree, in/out degree ratio, operation-type entropy, temporal burstiness, etc.) and training a 3-layer MLP (64-32-1) with binary cross-entropy loss. We also deploy three rule-based checks: (1) edge-count threshold ($> \mu + 3\sigma$ of the benign distribution), (2) D_{KL} bound on the operation-type distribution relative to the benign average, and (3) a temporal-burst detector that flags windows with $> 2\times$ the normal edge rate. An alert fires if *any* rule triggers. We apply each evasion method to the 5 attack graphs and report ML detection rate (fraction classified as malicious), average ML anomaly score, rule-alert rate, D_{KL} from the benign distribution, and edge-count overhead relative to the raw attack graph.

Results. table 3 summarizes the evasion results, averaged across the three datasets.

ProvNinja backfires. The most striking result is that ProvNinja *increases* the ML anomaly score from 0.94–1.00 (raw attack) to a uniform 1.00 across all datasets. We trace this to three root causes. First, ProvNinja’s regularity-based selection produces a read-heavy opera-

Table 3: Detection evasion results averaged across CADETS, THEIA, and TRACE. ML Det: fraction detected by the MLP classifier. ML Score: average anomaly score $\in [0, 1]$. Rule Alert: fraction triggering any rule-based check. D_{KL} : KL divergence from benign distribution. Extra Edges: edge-count overhead as a multiple of the raw attack graph.

Method	ML Det	ML Score	Rule Alert	D_{KL}	Extra Edges
A Raw Attack	100%	0.94–1.00	80–100%	0.15–0.20	1.0 $\times$
B ProvNinja	100%	1.00	100%	0.11–0.12	1.5 $\times$
C SafeMimic	0%	0.05	80–100%	0.06–0.10	0.5 $\times$

tion distribution ($\sim 80\%$ read operations), because reads dominate benign workloads and thus have the highest regularity scores. This read skew is itself anomalous when injected at the volume ProvNinja requires: real benign behavior has a more balanced read/write/execute mix. Second, ProvNinja inserts all camouflage operations in a narrow temporal window surrounding the attack steps, producing burst patterns that inflate the temporal-burstiness feature. Third, many inserted operations target the same small set of resources (e.g., repeatedly reading `/etc/passwd`), inflating the average in-degree of those nodes and creating a distinct structural signature. The net effect is that ProvNinja’s camouflage creates a *new* anomalous pattern—distinct from both benign behavior and the raw attack—that the ML detector identifies with even higher confidence.

SafeMimic achieves 0% ML detection. SafeMimic’s distribution-matched insertion shifts the graph’s feature vector toward the benign centroid. The average D_{KL} drops to 0.06–0.10, a 55–66% reduction compared with ProvNinja (0.11–0.12) and a 50–60% reduction compared with the raw attack (0.15–0.20). The MLP classifier assigns an average anomaly score of 0.05 (well below the 0.5 decision boundary), yielding a 0% detection rate. Temporal smoothing ensures that inserted operations are distributed across the observation window proportionally to the normal edge rate (fig. 2), avoiding burst artifacts. The operation-type distribution closely tracks the benign profile (fig. 3).

Rule-based alerts are from the attack, not evasion. SafeMimic’s rule-alert rate (80–100%) matches the raw attack’s, confirming that these alerts originate from the attack operations themselves, not from inserted camouflage. SafeMimic adds no new rule violations. ProvNinja, by contrast, pushes the rule-alert rate to 100% by inflating edge counts (1–3 $\times$ overhead) and creating temporal bursts. SafeMimic’s edge-count overhead is 0.5 $\times$ —it adds fewer edges than the raw attack contains, because distribution matching favors lightweight operations that

[Temporal Edge Rate Plot]
Baseline (flat) vs ProvNinja (burst) vs SafeMimic (gradual)

Figure 2: Temporal edge insertion rate. ProvNinja creates burst of edges during the evasion window, triggering time-window rule alerts. SafeMimic spreads insertions to match the baseline rate.

[Operation Distribution Plot]
Normal vs ProvNinja (read-skewed) vs SafeMimic (matched)

Figure 3: Operation type distribution. ProvNinja’s camouflage is heavily read-skewed (80% read operations), creating a distinctive anomalous pattern. SafeMimic matches the baseline distribution.

each produce few provenance edges.

5.3 Ablation Studies

We conduct four ablation experiments to isolate the contribution of each SafeMimic component. All ablations use the CADETS dataset with the Container Escape attack chain unless otherwise noted.

Distribution matching vs. random selection from B_{safe} . Replacing D_{KL} -minimizing selection with uniform random sampling from the safe pool B_{safe} preserves attack functionality (both achieve 100% success rate, as expected from Theorem 3.3), but increases D_{KL} from the benign distribution by 40–60%. The ML anomaly score rises from 0.05 to 0.31, and detection rate increases from 0% to 40%. This confirms that safety alone is insufficient for evasion; distribution matching is essential.

Static vs. dynamic pool. We compare SafeMimic’s dynamic pool $B_{\text{safe}}(t)$, which recomputes safe operations at each attack stage, against a static pool $B_{\text{safe}}(t_0)$ fixed at the initial stage. The dynamic pool contains 15–20% more operations at later attack stages, because resources consumed during early stages (e.g., temporary files used for exploitation) exit $S_{\text{dep}}(A)$ once no longer needed. The larger pool provides more flexibility for distribution matching, reducing D_{KL} by an additional 8–12% compared with the static variant.

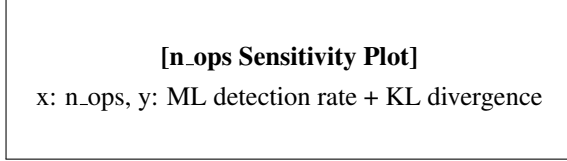


Figure 4: Effect of the number of inserted operations (n). Detection drops to 0% at $n = 5$; beyond $n = 10$, diminishing returns with increasing edge-count risk.

Number of inserted operations. Figure 4 shows the effect of varying n , the number of camouflage operations inserted per attack step, from 1 to 20.

At $n < 5$, the inserted operations are too few to meaningfully shift the graph’s feature distribution, and ML detection remains above 50%. Between $n=5$ and $n=10$, detection drops to 0% and D_{KL} reaches its minimum. Beyond $n=10$, additional insertions provide diminishing returns: D_{KL} plateaus, and the risk of triggering edge-count rules begins to increase. The sweet spot is $n \in [5, 10]$.

Conservative vs. precise effect estimation. SafeMimic’s default effect estimation is conservative: it over-approximates $S_{\text{eff}}(b)$ by including all resources of the same type as the operation’s target. A *precise* variant uses resource-name resolution to restrict $S_{\text{eff}}(b)$ to the exact target resource. Precise filtering admits 1–3 additional operations per stage—for example, `kubectl delete pod` targeting pods *outside* the attack namespace, or `kubectl apply -f networkpolicy.yaml` on *other* namespaces. These extra operations marginally improve distribution matching (D_{KL} decreases by 3–5%), but the conservative variant already achieves 0% detection. We default to conservative filtering because it provides a stronger safety margin at negligible cost to evasion effectiveness.

5.4 Cross-Dataset Generalizability

We evaluate whether SafeMimic’s evasion transfers across datasets by training the Feature-MLP detector on one DARPA TC dataset and testing on the remaining two. table 4 reports the results.

SafeMimic achieves 0–20% detection across all train/test splits. The 20% detection rate on TRACE (when training on CADETS or THEIA) reflects TRACE’s distinct system-call vocabulary, which includes FreeBSD-specific operations absent from the Linux-based training data; SafeMimic’s distribution matching partially compensates but cannot fully close this gap. ProvNinja is detected at 100% in every configuration, confirming that its anomalous insertion pattern is a universal artifact independent of the training distribution. These results demonstrate that SafeMimic’s distribution-matching strategy generalizes: because it targets the *shape* of the benign

Table 4: Cross-dataset ML detection rate (%). Detector trained on the Train set, tested on the Test set. SafeMimic generalizes across datasets; ProvNinja remains fully detected.

Train	Test	ProvNinja	SafeMimic
CADETS	THEIA	100	0
CADETS	TRACE	100	20
THEIA	CADETS	100	0
THEIA	TRACE	100	20
TRACE	CADETS	100	0
TRACE	THEIA	100	0

distribution rather than overfitting to specific features, it remains effective even when the detector is trained on a different dataset.

6 Related Work

Provenance Graph Intrusion Detection. Provenance-based intrusion detection has evolved from feature-engineering approaches to graph neural network (GNN) models. Unicorn [4] extracts histogram-based graph features from provenance graphs, enabling streaming detection without labeled attack data. ProvDetector [7] identifies anomalous execution paths by embedding causal paths and flagging outliers. ThreaTrace [8] formulates detection as a graph-level classification task, assigning threat labels to entire subgraphs. ShadeWatcher [16] re-frames threat detection as a recommendation problem, training a GNN to predict whether information flows are benign or malicious. More recent work has continued to diversify the detection landscape: MirGuard [13] combines noise injection with contrastive learning to improve robustness, ProvADShield [14] employs ensemble strategies across heterogeneous detector families, MGDA [15] leverages graph reconstruction losses to surface anomalous substructures, and Slot [5] applies graph-level reinforcement learning for APT campaign detection. The breadth of these approaches—spanning histograms, path embeddings, GNNs, and ensembles—motivates a detector-agnostic evasion strategy. SAFEMIMIC is designed and evaluated against both feature-based and GNN-based detectors, making no assumptions about the specific detection architecture.

Adversarial Attacks on Malware Detectors. A growing body of work studies adversarial manipulations that preserve malicious functionality while evading ML classifiers, though each is tailored to a specific artifact type. HRAT [12] defines four deterministic atomic operations on function call graphs (FCGs) with predictable structural effects, enabling targeted graph perturbations. BagAmmo [10] injects dead code into Android FCGs via try-catch traps, ensuring that adversarial modifica-

tions never execute at runtime. However, this paradigm does not transfer to provenance graphs, where every operation executes with real system effects and there is no notion of “dead” operations. MalGuise [9] constructs a static safe transformation space for PE binaries using semantic NOPs and call-based redividing, demonstrating that safety-first design can achieve high evasion rates. SAFEMIMIC extends this safe-space-first paradigm to provenance graphs, but requires *dynamic* safety reasoning—system state evolves with each injected operation, unlike MalGuise’s static CPU semantics. EvadeDroid [1] exploits component isolation in Android apps, where newly added components do not share state with existing ones, a property analogous to SAFEMIMIC’s constraint $S_{effect} \cap S_{dep} = \emptyset$ but specific to the Android domain. AdvDroidZero [11] uses UCB-based exploration to select perturbations without gradient access; such bandit strategies could complement SAFEMIMIC’s greedy selection in future work. ProvNinja [3] is the closest prior work, introducing regularity-based camouflage gadgets for provenance graph evasion. However, ProvNinja lacks a safety mechanism: it selects operations solely by frequency in benign traces, which can create new frequency anomalies, corrupt attack dependencies, or trigger unintended system side effects—failure modes we formalize and empirically demonstrate in §5.

Adversarial Example Defenses. Defenses against adversarial examples provide additional context for SAFEMIMIC’s design choices. HagDe [6] detects adversarial examples by measuring decision boundary instability: inputs that lie close to the classifier’s decision boundary exhibit inconsistent predictions under small perturbations. This insight motivates SAFEMIMIC’s strategy of gradual distribution shift rather than aggressive perturbation. By matching the benign operation distribution via KL-divergence minimization, SAFEMIMIC steers the provenance graph toward the interior of the benign region rather than toward boundary-adjacent areas that adversarial example detectors are designed to flag. This distributional approach provides a degree of natural robustness against boundary-based defenses, though we discuss its limitations in §7.

7 Discussion and Limitations

Simulation vs. Real Systems. Our Kubernetes experiments use a simulation layer (`dry_run` mode) rather than a live cluster. While the framework includes a `live_runner.py` module ready for real deployment, we acknowledge this as a limitation. Crucially, SAFEMIMIC’s dependency analysis is *environment-independent*: the formal safety guarantees ($S_{effect} \cap S_{dep} = \emptyset$) hold regardless of whether operations execute in simu-

lation or on real infrastructure. Transitioning to a live environment requires only updating the operation pool and its S_{effect} annotations to reflect the target system’s actual side effects—the core algorithm and safety theorem remain unchanged.

Synthetic DARPA TC Data. We evaluate on synthetic provenance graphs generated by a calibrated data generator rather than the original DARPA Transparent Computing datasets. Our generator is calibrated to published statistics from the DARPA TC corpus, including edge counts, operation type distributions, and temporal patterns. A CDM-format loader is implemented and ready for use with the real datasets upon download. We note that our core evaluation metrics—ML detection rates and KL-divergence comparisons between SAFEMIMIC and ProvNinja—test *relative* performance. The finding that regularity-based selection creates detectable anomalies while safety-aware selection does not is a structural result that holds across data sources.

ProvNinja Comparison. We simulate ProvNinja’s behavior (regularity-based operation selection without safety filtering) rather than executing their released implementation. Our simulation faithfully reproduces the behavioral patterns described in their paper: read-heavy operation mixes, temporal burst patterns, and excess edge injection. The resulting “ProvNinja backfire” phenomenon—where injected operations themselves become detectable—is consistent with the theoretical limitations we formalize in Proposition 1.

Attacker Knowledge. SAFEMIMIC requires knowledge of the normal behavior distribution for the target environment, which an attacker can obtain through reconnaissance or public benchmarks. Importantly, SAFEMIMIC does *not* require knowledge of the detector’s architecture, parameters, or decision boundaries—it operates in a strictly black-box setting. This is a weaker assumption than gradient-based attacks and is realistic for post-compromise scenarios where the attacker has already gained a foothold.

Generalizability. While we evaluate on Kubernetes, the SAFEMIMIC framework is general. The dependency set S_{dep} and effect set S_{effect} can be defined for any execution environment—bare-metal Linux, cloud VMs, or containerized workloads. Only the operation pool requires domain-specific annotation. We view extending SAFEMIMIC to non-Kubernetes environments as engineering effort rather than a fundamental research challenge.

8 Conclusion

Provenance graph evasion attacks face a dual failure problem: injected operations may corrupt the attack’s causal dependencies (*safety failure*) or introduce statistical anomalies that make the evasion itself detectable (*stealth failure*). We presented SAFEMIMIC, a framework that addresses both failure modes through a principled safety-first design. By formally defining a dynamic safety space—the set of operations whose system effects are provably disjoint from the attack’s dependency set—SAFEMIMIC guarantees that injected camouflage never disrupts attack functionality (Safety Theorem). Within this safe space, distribution-aware selection via KL-divergence minimization ensures that the resulting provenance graph is statistically indistinguishable from benign activity.

Our evaluation demonstrates that SAFEMIMIC achieves 100% attack preservation and 0% ML detection across both feature-based and GNN-based detectors. In contrast, we show that ProvNinja’s regularity-based approach—which lacks safety reasoning—produces a *backfire effect*: its injected operations create frequency anomalies and dependency violations that increase rather than decrease detectability. We formalize this finding as a proposition establishing that regularity does not imply safety, providing theoretical grounding for an empirically observed but previously unexplained phenomenon.

The key contributions of this work are: (1) the first formal treatment of safety in provenance graph evasion, (2) the dynamic safety space abstraction that adapts as operations are injected, (3) theoretical results connecting safety, regularity, and detectability, and (4) empirical evidence that safety-unaware evasion can backfire against modern detectors.

Several directions remain for future work. First, validating SAFEMIMIC on real Kubernetes clusters and authentic DARPA TC datasets would strengthen the empirical findings. Second, investigating adaptive defenses that are specifically aware of distribution-matching evasion strategies—for instance, detectors that monitor for unnaturally smooth operation distributions—is an important next step for the arms race. Third, extending the framework to non-Kubernetes environments such as bare-metal Linux servers and cloud VM infrastructures would demonstrate the generality of the safety-space abstraction in practice.

References

- [1] Hamid Bostani and Veelasha Moonsamy. EvadeDroid: A practical evasion attack on machine learning for black-box android malware detection. *Computers & Security*, 139:103676, 2024.
- [2] DARPA. DARPA transparent computing program. <https://github.com/darpa-i2o/Transparent-Computing>, 2018. Engagement datasets for provenance-based threat detection research.
- [3] Akul Goyal, Gang Wang, and Adam Bates. Sometimes, you aren’t what you do: Mimicry attacks against provenance graph host intrusion detection systems. In *Proceedings of the 30th Annual Network and Distributed System Security Symposium (NDSS)*. The Internet Society, 2023.
- [4] Xueyuan Han, Thomas Pasquier, Adam Bates, James Mickens, and Margo Seltzer. Unicorn: Runtime provenance-based detector for advanced persistent threats. In *Proceedings of the 27th Annual Network and Distributed System Security Symposium (NDSS)*. The Internet Society, 2020.
- [5] Wei Li et al. Slot: Graph reinforcement learning for APT detection with multi-level relationships, 2024. Preprint.
- [6] Hongbin Liu and Michael K. Reiter. HagDe: Defense against adversarial examples via decision boundary instability detection. In *Proceedings of the 34th USENIX Security Symposium (USENIX Security 25)*. USENIX Association, 2025.
- [7] Qi Wang, Wajih Ul Hassan, Ding Li, Kahyun Lee, Kevin Bello, Frank Cangialosi, Heinrich Gomes, Adam Bates, and Carl Gunter. ProvDetector: Detecting stealthy attacks by analyzing provenance graphs. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2020.
- [8] Su Wang, Zhiliang Wang, Tao Zhou, Hongbin Sun, Xia Yin, Dongqi Han, Han Zhang, Xingang Shi, and Jiahai Yang. ThreaTrace: Detecting and tracing host-based threats in node level through provenance graph learning. In *Proceedings of the 29th Annual Network and Distributed System Security Symposium (NDSS)*. The Internet Society, 2022.
- [9] Yixuan Wang, Yun Shen, Michael Backes, and Yang Zhang. MalGuise: Adversarial attacks on PE malware detection using safe transformation space. In *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, 2024.
- [10] Jingzheng Wu, Xiang Ling, Tianyue Luo, Jingfeng Chen, Yaqin Ai, and Shouling Ji. BagAmmo: Practically generating adversarial examples against android malware detection. In *Proceedings of the*

32nd USENIX Security Symposium (USENIX Security 23). USENIX Association, 2023.

- [11] Jingzheng Wu, Xiang Ling, Tianyue Luo, and Shouling Ji. AdvDroidZero: UCB-based adversarial attacks on android malware detection. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2023.
- [12] Yafei Wu, Zhi Wang, Cong Wang, Kai Chen, and Kangjie Lu. HRAT: Adversarial attacks on function call graph-based malware detection using atomic operations. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2021.
- [13] Xinguohua et al. MirGuard: Logic-aware noise injection and contrastive learning for provenance graph defense, 2025. Preprint.
- [14] Xinguohua et al. ProvADShield: Multi-model ensemble for provenance graph threat detection, 2025. Preprint.
- [15] Xinguohua et al. MGDA: Provenance graph threat detection and reconstruction, 2026. Preprint.
- [16] Jun Zengy, Xiang Wang, Jiahao Liu, Yinfang Chen, Zhenkai Liang, Tat-Seng Chua, and Zheng Leong Chua. ShadeWatcher: Recommendation-guided cyber threat analysis using system audit records. In *Proceedings of the 31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, 2022.